Министерство науки и высшего образования РФ
Федеральное государственное бюджетное образовательное учреждение высшего образования
«Уфимский университет науки и технологий»

Кафедра: Высокопроизводительных вычислений и дифференциальных уравнений

ИССЛЕДОВАНИЕ ДИНАМИКИ ОДНОМЕРНОЙ ЦЕПОЧКИ ЧАСТИЦ С ПОТЕНЦИАЛОМ МЕЖЧАСТИЧНОГО ВЗАИМОДЕЙСТВИЯ ФЕРМИ-ПАСТА-УЛАМА

ОТЧЕТ

к лабораторной работе №3 по дисциплине

«Математическое моделирование»

Группа МКН-418Б	Фамилия И. О.	Подпись	Дата	Оценка
Студент	Халитова А.А.			
Принял	Феоктистов Б.А.			

Уфа 2026

Содержание

1 Введение **3**

2 Теоретическая часть **4**
2.1 Алгоритм Верле в скоростной форме. 4
2.2 Симплектический алгоритм типа Верле в скоростной форме. 5

3 Постановка задачи **6**

4 Практическая часть **7**
4.1 Начальные условия 7
4.2 Уравнение движения 7
4.3 Сравнение точности алгоритма посредством сравнения величины полной энергии системы (гамильтониана) 7
4.4 Поиск солитонов FPU-α потенциала 8
4.5 Поиск солитонов FPU-β потенциала 9

5 Вывод **10**

6 Листинг **11**

1 Введение

Цель работы: получить навык моделирования динамики системы многих частиц методами молекулярной динамики на примере задачи распространения возмущений в одномерной цепочке частиц одинаковой массы, связанных нелинейным потенциалом взаимодействия Ферми-Паста-Улама.

2 Теоретическая часть

Рассматривается цепочка частиц одинаковой массы, находящаяся в начальный момент времени в состоянии равновесия (расстояния между частицами одинаковы). В качестве граничных условий используется условие периодичности (или условной замкнутости) цепочки. В качестве потенциала межчастичного взаимодействия рассматривается нелинейный степенной потенциал типа Ферми–Паста–Улама (FPU-потенциал):

$$V(r) = \frac{r^2}{2} + \alpha\frac{r^3}{3} + \beta\frac{r^4}{4}. \tag{1}$$

При $\alpha \neq 0$, $\beta = 0$ данный потенциал принято называть FPU-α, при $\alpha = 0$, $\beta \neq 0$ – FPU-β.

Уравнение движения i-й частицы будет иметь вид:

$$m\frac{d^2 q_i}{dt^2} = -\nabla V(q_i) \equiv -\frac{\partial}{\partial q_i}\left[V(q_{i+1} - q_i) + V(q_i - q_{i-1})\right]. \tag{2}$$

Правая часть этого уравнения представляет собой сумму двух сил, действующих на i-ю частицу со стороны соседних $(i-1)$-й и $(i+1)$-й частиц. Величина q_i – смещение i-й частицы от положения равновесия.

Для интегрирования уравнений движения вида (2) используются два численных алгоритма.

2.1 Алгоритм Верле в скоростной форме.

Основные шаги алгоритма на каждом временном шаге:

$$\begin{aligned}
&q_i(t+\Delta t) = q_i(t) + v_i(t)\Delta t + \frac{1}{2}a_i(t)\Delta t^2;\\
&v_i(t+\Delta t/2) = v_i(t) + \frac{1}{2}a_i(t)\Delta t;\\
&a_i(t+\Delta t) = -\frac{1}{m}\nabla V(q_i(t+\Delta t));\\
&v_i(t+\Delta t) = v_i(t+\Delta t/2) + \frac{1}{2}a_i(t+\Delta t)\Delta t.
\end{aligned}$$

Здесь $v_i = \dfrac{dq_i}{dt}$ и $a_i = \dfrac{d^2 q_i}{dt^2}$ – скорость и ускорение i-й частицы соответственно. В начальный момент времени $t = 0$ значения $q_i(0)$ и $v_i(0)$ известны, и по ним рассчитываются значения $a_i(0)$.

2.2 Симплектический алгоритм типа Верле в скоростной форме.

Основные действия алгоритма на каждом временном шаге:

$$
\begin{aligned}
&q_i^{(1)}(t) = q_i(t) + v_i(t)\,\xi\Delta t;\\
&a_i^{(1)}(t) = -\frac{1}{m}\nabla V(q_i^{(1)}(t));\\
&v_i^{(1)}(t) = v_i(t) + \frac{1}{2}a_i^{(1)}(t)\Delta t;\\
&q_i^{(2)}(t) = q_i^{(1)}(t) + v_i^{(1)}(t)\,(1-2\xi)\Delta t;\\
&a_i^{(2)}(t) = -\frac{1}{m}\nabla V(q_i^{(2)}(t));\\
&v_i(t+\Delta t) = v_i^{(1)}(t) + \frac{1}{2}a_i^{(2)}(t)\Delta t;\\
&q_i(t+\Delta t) = q_i^{(2)}(t) + v_i(t+\Delta t)\,\xi\Delta t.
\end{aligned}
$$

Здесь параметр ξ определяется из условия минимальности значения коэффициента при Δt^3 в оценке погрешности метода и равен:

$$
\xi = \frac{1}{2} - \frac{\sqrt[3]{2\sqrt{326}+36}}{12} + \frac{1}{6\sqrt[3]{2\sqrt{326}+36}} \approx 0.1931833275037836.
$$

3 Постановка задачи

В рамках данной работы необходимо выполнить следующие задачи:

1. Определить вид правой части уравнения (2) для потенциала (1).

2. На языке программирования C++ выполнить программную реализацию приведенных численных алгоритмов Верле и симплектического типа Верле.

3. Провести сравнение точности алгоритмов посредством сравнения величины полной энергии системы (гамильтониана)

$$H = \sum_{i=0}^{n} \left[\frac{mv_i^2}{2} + V(q_{i+1} - q_i) \right]$$

   после N временных шагов. Количество частиц n, параметры α и β потенциала взаимодействия, а также значение N задаются преподавателем.

4. Для FPU-α потенциала взаимодействия определить диапазон значений $\alpha \in (0, 1)$, в котором в пространстве скоростей существуют односолитонные решения при условии, что возмущение задается в центре цепочки в виде приложенных к двум соседним частицам импульсов, равных по величине и противоположных по направлению (так что суммарный импульс системы равен нулю). Выполнить анимационную визуализацию динамики цепочки.

5. Для FPU-β потенциала взаимодействия определить диапазоны значений $\beta > 0$, в которых в пространстве скоростей существуют одно-, двух- и трехсолитонные решения. Возмущения задаются аналогично п. 4. Выполнить анимационную визуализацию цепочки.

4 Практическая часть

4.1 Начальные условия

Задание выполнялось с параметрами для 19 варианта.
При $t = 0$:

$$q_i = 0, i \in (0, 2N+1), i \notin N, N+1,$$

$$q_N = -q, q_{N+1} = q, q = 0.01I.$$

где I - номер варианта.
Масса каждой частицы равна m = 1.

4.2 Уравнение движения

Уравнение движения (2) для потенциала (1) будет иметь вид:

$$m\frac{d^2 q_i}{dt^2} = -\nabla V(q_i) \equiv -\Big[\frac{\partial V(q_i - q_{i-1})}{\partial q_i} + \frac{\partial V(q_{i+1} - q_i)}{\partial q_i}\Big]. \tag{3}$$

Производная потенциала (1) имеет вид:

$$V'(r) = r + \alpha r^2 + \beta r^3.$$

Вычисляя частные, получим:

$$\frac{\partial V(q_i - q_{i-1})}{\partial q_i} = V'(q_i - q_{i-1}) = (q_i - q_{i-1}) + \alpha(q_i - q_{i-1})^2 + \beta(q_i - q_{i-1})^3,$$
$$\frac{\partial V(q_{i+1} - q_i)}{\partial q_i} = -V'(q_{i+1} - q_i) = -(q_{i+1} - q_i) - \alpha(q_{i+1} - q_i)^2 - \beta(q_{i+1} - q_i)^3.$$

Подставляя в уравнение 3, получим:

$$m\frac{d^2 q_i}{dt^2} = (q_{i+1} - 2q_i + q_{i-1}) + \alpha\Big[(q_{i+1} - q_i)^2 - (q_i - q_{i-1})^2\Big]$$
$$+ \beta\Big[(q_{i+1} - q_i)^3 - (q_i - q_{i-1})^3\Big].$$

4.3 Сравнение точности алгоритма посредством сравнения величины полной энергии системы (гамильтониана)

Для проверки выполнения закона сохранения энергии для обоих алгоритмов возьмем n = 1000, Δt = 0.01, $\alpha = 1$, $\beta = 1$.

Таблица 1 — Отклонения энергии системы от начального значения

Алгоритм	Количество итераций		
	10^5	10^6	10^7
Верле	6.9×10^{-6}	6.31×10^{-6}	6.73×10^{-6}
Симплектический Верле	2.99×10^{-7}	2.96×10^{-7}	2.94×10^{-7}

Симплектический алгоритм типа Верле в скоростной форме является более точным, чем алгоритм Верле в скоростной форме, однако у него же уходит больше времени на вычисления. Оба алгоритма накапливают погрешность в ходе вычислений.

Время вычислений:

Таблица 2 — Время вычислений (в секундах)

Алгоритм	Количество итераций		
	10^5	10^6	10^7
Верле	8.68	87.09	876.21
Симплектический Верле	16.86	168.34	2190.65

4.4 Поиск солитонов FPU-α потенциала

Односолитонное решение существует при $\alpha \in (0.1,2)$.

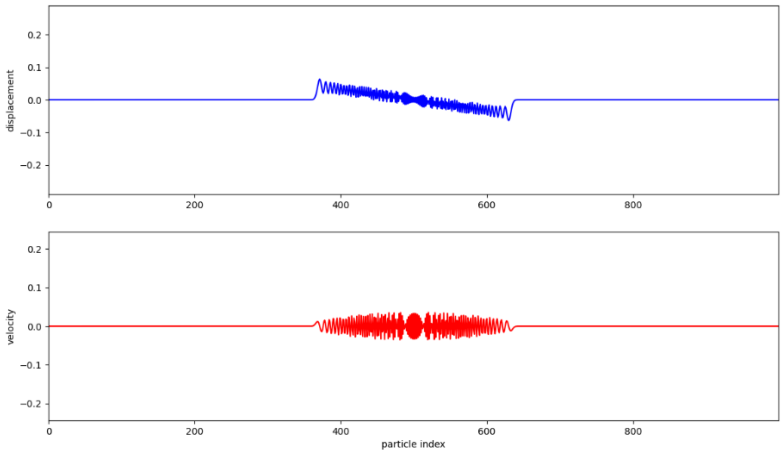

Рисунок 1 — Односолитонное решение при $\alpha = 0.8$

4.5 Поиск солитонов FPU-β потенциала

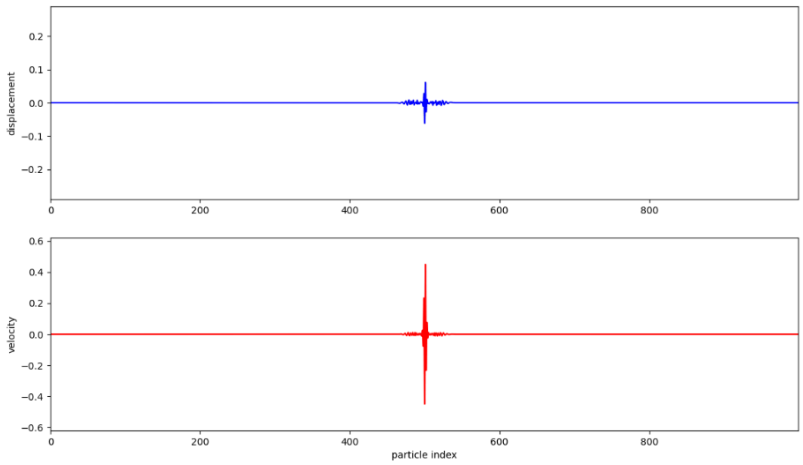

Рисунок 2 — Односолитонное решение при $\beta = 30$

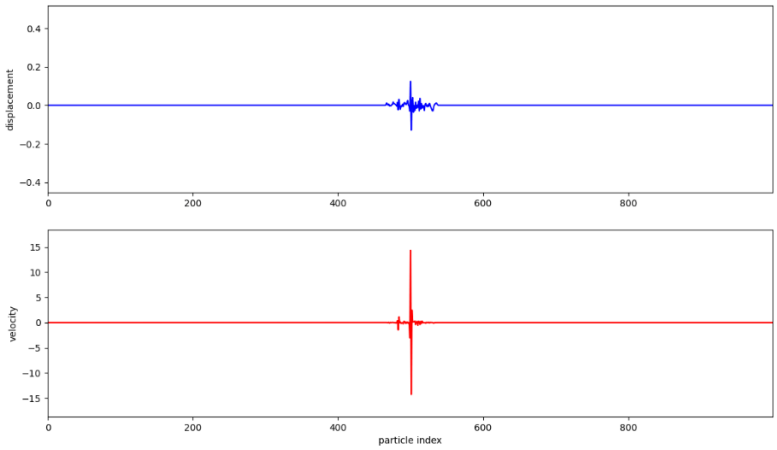

Рисунок 3 — Двухсолитонное решение при $\beta = 47000.05$

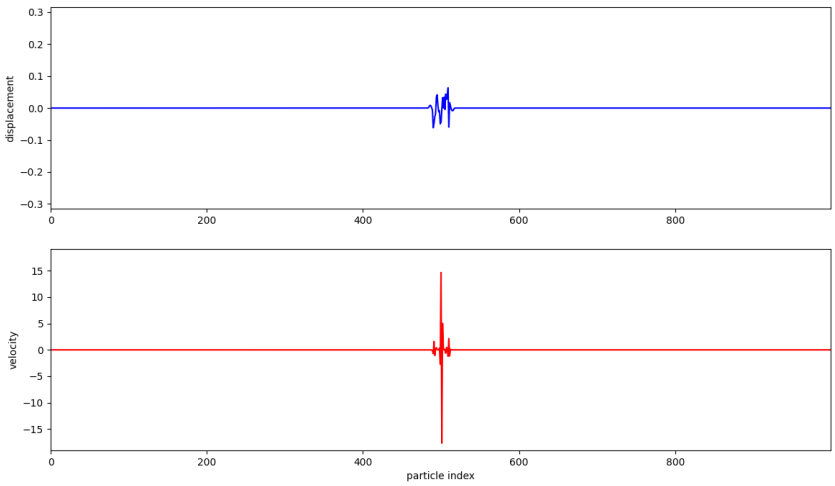

Рисунок 4 — Трехсолитонное решение при $\beta = 49500$

5 Вывод

В ходе лабораторной работы были получены навыки моделирования динамики системы многих частиц методами молекулярной динамики на примере задачи распространения возмущений в одномерной цепочке частиц одинаковой массы, связанных нелинейным потенциалом взаимодействия типа Ферми-Паста-Улама. Было проведено сравнение алгоритмов интегрирования движения частиц и сделан вывод о точности алгоритмов. Симплектический алгоритм типа Верле в скоростной форме точнее алгоритма Верле в скоростной форме на два порядка, но дольше работает. Визуализирована динамика частиц и найдены такие параметры α, β , при которых возникают одно, двух- и трехсолитонные решения.

6 Листинг

main.cpp

```cpp
#include <chrono>
#include <cmath>
#include <ctime>
#include <fstream>
#include <iomanip>
#include <iostream>
#include <memory>
#include <vector>

using namespace std;

class FPUChain {
        private:
        int num_chain;
        double alpha, beta, mass, dt;
        const double xi = 0.1931833275037836;
        long long total_time;

        vector<double> q;  // displacement
        vector<double> v;  // velocity
        vector<double> a;  // acceleration

        public:
        FPUChain(int n, double alpha, double beta, double mass, double time_step, long long total_time)
        : num_chain(n),
        alpha(alpha),
        beta(beta),
        mass(mass),
        dt(time_step),
        total_time(total_time) {
                q.resize(num_chain, 0.0);
                v.resize(num_chain, 0.0);
                a.resize(num_chain, 0.0);
        }

        double fpu(double r) {
                return 0.5 * r * r + (alpha / 3.0) * r * r * r + (beta / 4.0) * r * r * r * r;
        }

        double fpu_deriv(double r) { return r + alpha * r * r + beta * r * r * r; }

        int prev_index(int i) const { return (i - 1 + num_chain) % num_chain; }

        int next_index(int i) const { return (i + 1) % num_chain; }

        double comput_force(int i) {
                int prev = prev_index(i);
                int next = next_index(i);

```

```
                double r_right = q[next] - q[i];
                double r_left = q[i] - q[prev];

                double force_right = fpu_deriv(r_right);
                double force_left = fpu_deriv(r_left);

                double force = force_right - force_left;
                return force;
        }

        void init_soliton_displacement(double q0) {
                fill(q.begin(), q.end(), 0.0);
                fill(v.begin(), v.end(), 0.0);

                int center = num_chain / 2;
                if (center + 1 < num_chain) {
                        q[center] = -q0;
                        q[center + 1] = q0;
                }

                for (int i = 0; i < num_chain; i++) {
                        a[i] = comput_force(i) / mass;
                }
        }

        void init_soliton_impulse(double v0) {
                fill(q.begin(), q.end(), 0.0);
                fill(v.begin(), v.end(), 0.0);

                int center = num_chain / 2;
                if (center + 1 < num_chain) {
                        v[center] = v0;
                        v[center + 1] = -v0;
                }

                for (int i = 0; i < num_chain; i++) {
                        a[i] = comput_force(i) / mass;
                }
        }

        double velocity_verlet(long long num_steps, bool save_to_file = false,
        const string& filename = "velocity_verlet.csv") {
                ofstream f_out;
                if (save_to_file) {
                        f_out.open(filename);
                        f_out << "step,time";
                        for (int i = 0; i < num_chain; i++) {
                                f_out << ",q" << i << ",v" << i;
                        }
                        f_out << "\n";

                        f_out << "0,0.0";
                        for (int i = 0; i < num_chain; i++) {
                                f_out << "," << scientific << setprecision(12) << q[i] << "," << v[i];
                        }
```

```
                        f_out << endl;
                }

                for (long long step = 1; step <= num_steps; ++step) {
                        for (int i = 0; i < num_chain; i++) {
                                q[i] = q[i] + v[i] * dt + 0.5 * a[i] * dt * dt;
                                v[i] = v[i] + 0.5 * a[i] * dt;
                        }

                        for (int i = 0; i < num_chain; i++) {
                                a[i] = comput_force(i) / mass;
                        }

                        for (int i = 0; i < num_chain; i++) {
                                v[i] = v[i] + 0.5 * a[i] * dt;
                        }

                        if (save_to_file && step % 100 == 0) {
                                double time = step * dt;
                                f_out << step << "," << time;
                                for (int i = 0; i < num_chain; i++) {
                                        f_out << "," << scientific << setprecision(12) << q[i] << "," << v[i];
                                }
                                f_out << "\n";
                        }
                }

                double E_final = hamilton();

                if (save_to_file) {
                        f_out.close();
                }

                return E_final;
        }

        double simplex_velocity_verle(long long num_steps, bool save_to_file = false,
        const string& filename = "symplectic_verlet.csv") {
                ofstream f_out;
                if (save_to_file) {
                        f_out.open(filename);
                        f_out << "step,time";
                        for (int i = 0; i < num_chain; i++) {
                                f_out << ",q" << i << ",v" << i;
                        }
                        f_out << "\n";

                        f_out << "0,0.0";
                        for (int i = 0; i < num_chain; i++) {
                                f_out << "," << scientific << setprecision(12) << q[i] << "," << v[i];
                        }
                        f_out << "\n";
```

```
                }

                for (long long step = 1; step <= num_steps; ++step) {
                        for (int i = 0; i < num_chain; i++) {
                                q[i] = q[i] + v[i] * xi * dt;
                        }

                        for (int i = 0; i < num_chain; i++) {
                                a[i] = comput_force(i) / mass;
                        }

                        for (int i = 0; i < num_chain; i++) {
                                v[i] = v[i] + 0.5 * a[i] * dt;
                        }

                        for (int i = 0; i < num_chain; i++) {
                                q[i] = q[i] + v[i] * (1.0 - 2.0 * xi) * dt;
                        }

                        for (int i = 0; i < num_chain; i++) {
                                a[i] = comput_force(i) / mass;
                        }

                        for (int i = 0; i < num_chain; i++) {
                                v[i] = v[i] + 0.5 * a[i] * dt;
                        }

                        for (int i = 0; i < num_chain; i++) {
                                q[i] = q[i] + v[i] * xi * dt;
                        }

                        if (save_to_file && step % 100 == 0) {
                                double time = step * dt;
                                f_out << step << "," << time;
                                for (int i = 0; i < num_chain; i++) {
                                        f_out << "," << scientific << setprecision(12) << q[i] << "," << v[i];
                                }
                                f_out << endl;
                        }
                }

                double E_final = hamilton();

                if (save_to_file) {
                        f_out.close();
                }

                return E_final;
        }

        double hamilton() {
                double energy = 0.0;

                for (int i = 0; i < num_chain; i++) {
                        energy += 0.5 * mass * v[i] * v[i];
```

```
                        int next = next_index(i);
                        energy += fpu(q[next] - q[i]);
                }

                return energy;
        }

        void reset() {
                fill(q.begin(), q.end(), 0.0);
                fill(v.begin(), v.end(), 0.0);
                fill(a.begin(), a.end(), 0.0);
        }

        void compare_methods(long long num_steps, double q0) {
                cout << "num steps = " << num_steps << endl;
                cout << "Velocity Verlet" << endl;
                init_soliton_displacement(q0);
                double E0 = hamilton();
                auto start_vv = chrono::high_resolution_clock::now();
                double E_VV = velocity_verlet(num_steps, false, "velocity_verlet_energy.csv");
                auto end_vv = chrono::high_resolution_clock::now();

                auto duration_vv = chrono::duration_cast<chrono::milliseconds>(end_vv - start_vv);

                cout << "final energy: " << E_VV << endl;
                cout << "execution time: " << duration_vv.count() / 1000.0 << " s" << endl;

                cout << "Symplectic Velocity Verlet" << endl;
                reset();
                init_soliton_displacement(q0);
                auto start_sv = chrono::high_resolution_clock::now();
                double E_VVV = simplex_velocity_verle(num_steps, false, "symplectic_verlet_energy.csv");
                auto end_sv = chrono::high_resolution_clock::now();

                auto duration_sv = chrono::duration_cast<chrono::milliseconds>(end_sv - start_sv);
                cout << "final energy: " << E_VVV << endl;
                cout << "execution time: " << duration_sv.count() / 1000.0 << " s" << endl;

                cout << "Velocity Verlet error = " << scientific << abs(E0 - E_VV) << endl;
                cout << "Symplectic Verlet error = " << scientific << abs(E0 - E_VVV) << endl;
                cout << "\n\n\n";
        }
};

int main() {
        {
                double alpha = 1.0;
                double beta = 1.0;
```

```
                int n = 1000;
                double dt = 0.01;
                long long total_steps = 1e6;
                double q0 = 0.19;
                double mass = 1.0;

                unique_ptr<FPUChain> chain = make_unique<FPUChain>(n, alpha, beta, mass, dt, total_steps);

                chain->compare_methods(total_steps, q0);
                chain->compare_methods(total_steps * 10, q0);
                chain->compare_methods(total_steps * 100, q0);
        }

        // {
                //      double alpha = 0.0;
                //      double beta = 4000.0;
                //      int n = 1000;
                //      double dt = 0.01;
                //      long long total_steps = 1e4;
                //      double q0 = 0.19;
                //      double mass = 1.0;

                //      unique_ptr<FPUChain> chain = make_unique<FPUChain>(n, alpha, beta, mass, dt,
                //      total_steps);

                //      chain->init_soliton_displacement(q0);
                //      auto res = chain->velocity_verlet(total_steps, true,
                //      "../labs/lr3/src/velocity_verlet.csv");

                //      cout << "python part" << endl;
                //      system("python ../labs/lr3/src/main.py");
                // }
        return 0;
}
```

main.py

```
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from matplotlib.animation import FuncAnimation

df = pd.read_csv("../labs/lr3/src/velocity_verlet.csv")

steps = df["step"].values
time = df["time"].values
q_data = df[[col for col in df.columns if col.startswith("q")]].values
v_data = df[[col for col in df.columns if col.startswith("v")]].values

n_particles = q_data.shape[1]

skip_frames = max(1, len(steps) // 200)
indices = np.arange(0, len(steps), skip_frames)
steps, time, q_data, v_data = (
steps[indices],
```

```
time[indices],
q_data[indices],
v_data[indices],
)
n_frames = len(indices)

fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(14, 8))

(line_q,) = ax1.plot([], [], "b-", linewidth=1.5)
(line_v,) = ax2.plot([], [], "r-", linewidth=1.5)
time_text = fig.text(
0.02, 0.98, "", fontsize=14, bbox=dict(boxstyle="round", facecolor="lightblue")
)

x_range = [0, n_particles - 1]
ax1.set(
xlim=x_range,
ylim=[np.min(q_data) - 0.1, np.max(q_data) + 0.1],
ylabel="displacement",
)
ax2.set(
xlim=x_range,
ylim=[np.min(v_data) - 0.1, np.max(v_data) + 0.1],
ylabel="velocity",
xlabel="particle index",
)


def animate(frame):
x = np.arange(n_particles)
line_q.set_data(x, q_data[frame])
line_v.set_data(x, v_data[frame])
time_text.set_text(f"t = {time[frame]:.2f}")
return line_q, line_v, time_text


anim = FuncAnimation(fig, animate, frames=n_frames, interval=200, blit=True)
anim.save("../labs/lr3/src/fpu_animation.gif", writer="pillow", dpi=100)
```